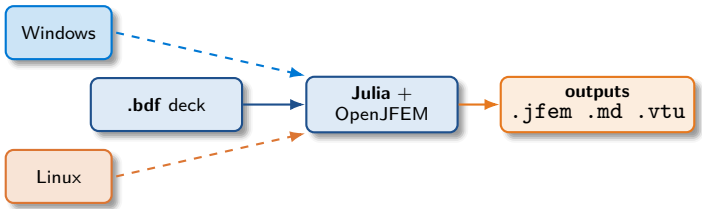
Running the JFEM Solver

User reference guide for Windows and Linux
Launch options, output formats, batch runs, sysimage

JFEM 2026 documentation

JFEM 2026 workspace

May 2026



Outline

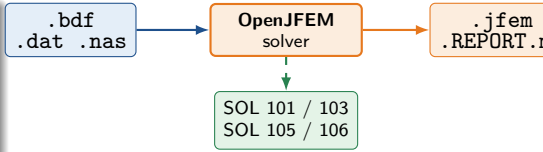
What JFEM does — 60-second overview

Solver scope

JFEM (`OpenJFEM`) is a Julia finite-element solver with a **Nastran-compatible** BDF front end. Supported solution sequences:

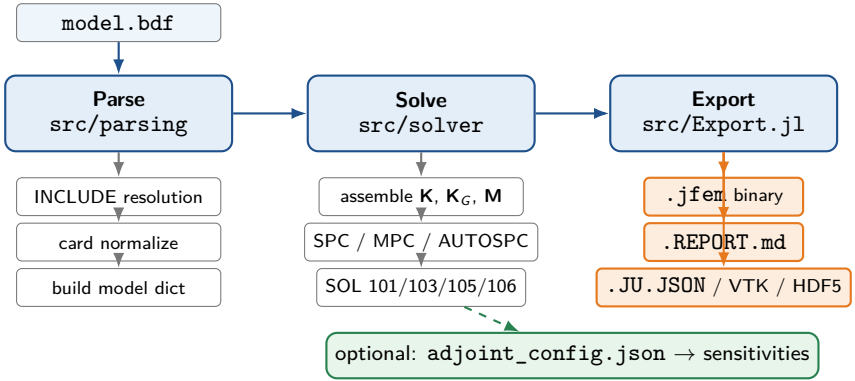
- `SOL 101` — linear static
- `SOL 103` — normal modes
- `SOL 105` — linear buckling
- `SOL 106` — nonlinear static (experimental)

It reads `.bdf`, `.dat`, or `.nas` decks and writes text/binary results plus a Markdown report.



Designed for **batch buckling** and **Python-driven sizing loops**, with first-class precompilation and sysimage support.

Three-stage execution model: **parse** → **solve** → **export**



Everything you launch ultimately calls `OpenJFEM.main(deck;...)` which executes these three stages in order.

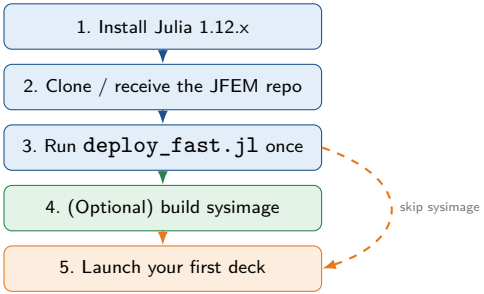
Prerequisites and first-time setup

What you need

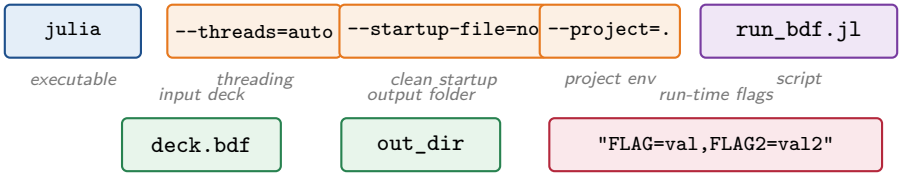
- **Julia 1.12.x** on PATH (or set `JULIA=/path/to/julia`)
- **Git** (only for cloning / pulling JFEM)
- A **bulk-data deck**: `.bdf`, `.dat`, or `.nas`
- A working directory for outputs

Disk / memory rule of thumb

- First precompile: ∼3–5 min
- Sysimage build: +5–10 min, ∼300 MB
- Solve: dominated by SOL type and mesh size



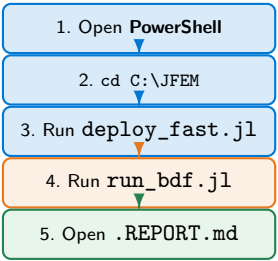
Anatomy of a JFEM launch command



Read left to right

Blue = Julia executable. **Orange** = Julia flags that should not change. **Purple** = the JFEM tool you call. **Green** = your data (input deck, output folder). **Red** = a comma-separated string of `JFEM_*` environment flags — keep the quotes.

Launching on Windows — PowerShell



First-time precompile (once after install/update):

```
cd C:\JFEM
julia --threads=auto --startup-file=no `
      --project=. .\tools\deploy_fast.jl
```

Single buckling case:

```
julia --threads=auto --startup-file=no `
      --project=. .\tools\testing\run_bdf.jl `
      C:\models\panel.bdf D:\runs\panel `
      "JFEM_EXPORT_BINARY=false,JFEM_SUPPRESS_THREAD_HINT=1"
```

The PowerShell backtick ` continues the command on the next line. On a single line, omit it.

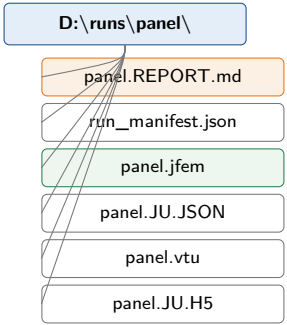
Windows: where outputs go

Output folder layout

After a successful run the output directory contains:

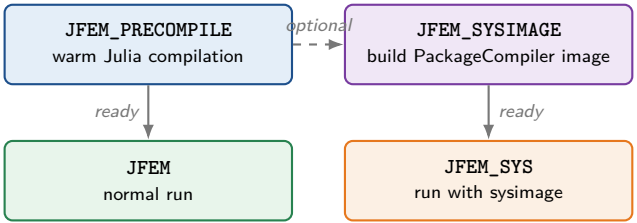
```
D:\runs\panel\
  run_manifest.json # what was launched
  panel.REPORT.md # human-readable report
  panel.jfem # binary (if enabled)
  panel.JU.JSON # results JSON (if enabled)
  panel.vtu # VTK (if enabled)
  panel.JU.H5 # HDF5 (if enabled)
```

The Markdown report is the main file to open first: model counts, active flags, solver timings, and (for SOL 105) the buckling load factors.



only the toggles you enable are written

Launching on Linux — four Bash helpers



Source these helpers first:

```
source jfem_linux_
bash_functions.sh
```

Then `JFEM /path/to/deck.bdf`

File location

`01_PROJECT_FOLDER/DOCUMENT_SOURCES/Software_Installation/bash_scripts/jfem_linux_bash_functions`

Copy or source this file from your `~/.bashrc` (or `.bash_aliases`) so the four functions are available in any shell.

Linux: typical usage of the four helpers

First-time setup (one-time):

```
source /path/to/jfem_linux_bash_functions.sh
JFEM_PRECOMPILE
```

Better precompile (representative production deck):

```
JFEM_PRECOMPILE /projects/.../MODELS/VTP105.bdf
```

Optional sysimage:

```
JFEM_SYSIMAGE /projects/.../MODELS/VTP105.bdf
```

Run a deck:

```
JFEM /projects/.../MODELS/VTP105.bdf # normal
JFEM_SYS /projects/.../MODELS/VTP105.bdf # with sysimage
JFEM_SYS /projects/.../MODELS/VTP105.bdf /runs/VTP105 # custom output
```

The helpers default to a hard-coded `jfem_dir`. Edit the variable inside each function if your install path differs.

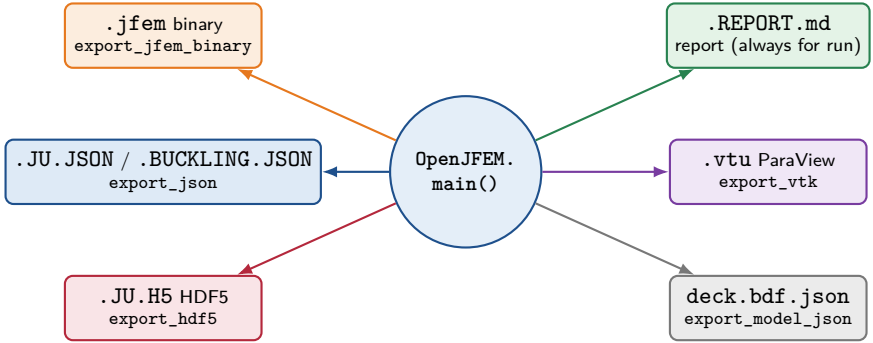
Side-by-side: Windows vs. Linux commands

Step	Windows PowerShell	Linux Bash (helpers loaded)
First precompile	`julia ... .\tools\deploy_fast.jl`	`JFEM_PRECOMPILE`
Precompile with deck	`... deploy_fast.jl --deck C:\m.bdf`	`JFEM_PRECOMPILE /m.bdf`
Build sysimage	`... deploy_fast.jl --sysimage=... --install-packagecompiler`	`JFEM_SYSIMAGE /m.bdf`
Single run	`... .\tools\testing\run_bdf.jl deck.bdf out\dir "flags"`	`JFEM /m.bdf [/out/dir]`
Single run + sysimage	`julia --sysimage build\OpenJFEM_sysimage.dll ... run_bdf.jl ...`	`JFEM_SYS /m.bdf [/out/dir]`
Batch run	`... .\tools\run_batch_manifest.jl cases.json --quiet`	`julia ... ./tools/run_batch_manifest.jl cases.json --quiet`

One mental model

The Linux helpers are convenience wrappers around the same Julia commands you type on Windows. They hide the long Julia command line behind four short function names.

Output format selection — the six toggles



Default in the Linux JFEM / JFEM_SYS helpers

`export_jfem_binary=true`, `export_report=true`, all others `false`. Edit the function body to flip a toggle, or pass the right options through a manifest.

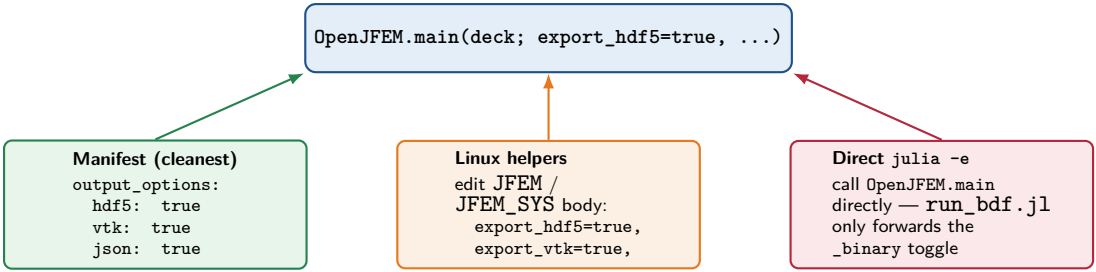
Output formats — when to use which

Format	Extension	When to use
JFEM binary	`.jfem`	Full results in compact form. Reload with the `POST/postv11.html` browser viewer. Set `JFEM_EXPORT_BINARY=false` to skip for max throughput.
Markdown report	`.REPORT.md`	Always start here. Lists timing, model counts, active flags, and (SOL 105) buckling load factors.

Results JSON	`.JU.JSON`
	`.BUCKLING.JSON`
	`.NONLINEAR.JSON`
		Machine-readable results for Python, MATLAB, or downstream sizing tools. Name depends on the SOL.

ParaView VTK	`.vtu`	Visual checks: mode shapes, displacements, stress fields. Skip for headless production.
HDF5	`.JU.H5`	Large structured results with attributes; ideal for archival or HPC pipelines.
Model JSON	`deck.bdf.json`	A snapshot of the parsed/built model before solving — useful for debugging the BDF interpretation.
Card inventory	`.cards.json`	Audit of every Nastran card recognized in the deck; helpful when migrating decks from other solvers.

Enabling HDF5, VTK, and results JSON



The underlying knob

All three paths ultimately call `OpenJFEM.main(deck; ...)` with the boolean keywords `export_hdf5`, `export_vtk`, `export_json`, `export_jfem_binary`, `export_report`, `export_model_json`, `export_card_inventory`. All default to `false` except `export_jfem_binary` (and `export_report` in the helpers).

Three ways to turn on HDF5 — worked examples

1. JSON batch manifest (preferred for repeatable runs):

```
"defaults": {
  "output_options": {
    "binary": false,
    "json": true,
    "hdf5": true,
    "vtk": true,
    "report": true
  }
}
```

Aliases accepted by the manifest reader: `hdf5`, `h5`, `export_hdf5` / `vtk`, `export_vtk` / `json`, `export_json`, `results_json`.

Caveat: if `eigenvalues_only: true` is set for SOL 105, HDF5 and VTK are forced off.

2. Edit the Linux helper body (`jfem_linux_bash_functions.sh`):

```
OpenJFEM.main(input_file;
    output_dir=output_dir,
    export_json=true,
    export_vtk=true,
    export_hdf5=true, # <-- flip to true
    export_jfem_binary=true,
    export_report=true,
)
```

Run-time environment flags (JFEM_*)

Flag	Default	Effect
JFEM_EXPORT_BINARY	true	false skips .jfem for max solver throughput.
JFEM_MATRIX_ASYMMETRY_CHECK	true	false skips an expensive buckling diagnostic.
JFEM_SOL105_STORE_PUBLIC_MODE_SHAPES	true	false drops the duplicate mode-shape array from the returned dict.
JFEM_SOL105_EIGENVALUES_ONLY	false	true keeps only the eigenvalues (no mode vectors).
JFEM_SUPPRESS_THREAD_HINT	0	1 suppresses the single-thread warning in logs.
JULIA (shell var)	—	Path to a non-default Julia executable.

Recommended *fast* string (single-case CLI)

```
"JFEM_EXPORT_BINARY=false,JFEM_MATRIX_ASYMMETRY_CHECK=false,JFEM_SOL105_STORE_PUBLIC_MODE_SHAPES=false,JF
```

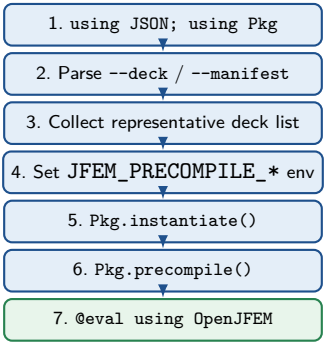
Precompilation — why and how

Why precompile

Julia compiles methods the first time it sees each new combination of types and code paths. Without precompilation, the *first* JFEM run pays minutes of compile time. `deploy_fast.jl` warms those paths upfront so later runs are fast.

What it touches

- `Pkg.instantiate()` — restore dependencies
- `Pkg.precompile()` — compile all packages
- Loads `using OpenJFEM` and runs a representative workload



deploy_fast.jl options

Option	Meaning
`--deck <path>`	Add one representative deck to the precompile workload.
`--manifest <path>`	Use every `input` listed in a JSON batch manifest.
`--flags FLAG=val,...`	Override `JFEM_*` env values during precompile.
`--sysimage=<path>`	Also build a PackageCompiler sysimage at this path.
`--install-packagecompiler`	Permit `Pkg.add("PackageCompiler")` if absent.

Three precompile sources (with no `--deck`, bundled tiny decks are used):

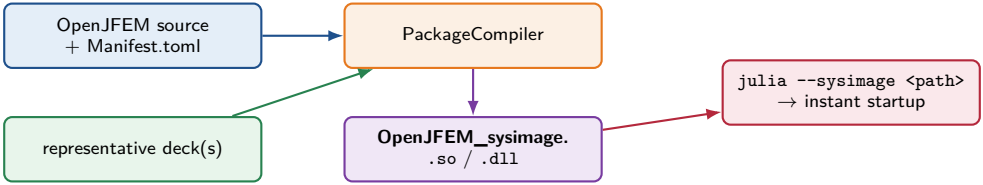
```
# 1. Default (bundled examples/precompile/*.bdf)
julia ... .\tools\deploy_fast.jl

# 2. Representative deck
julia ... .\tools\deploy_fast.jl --deck C:\models\VTP105.bdf

# 3. Manifest
julia ... .\tools\deploy_fast.jl --manifest C:\models\cases.json
```

Rule of thumb: use a deck that *resembles* the cases you will run in production.

Sysimage — ahead-of-time compilation



What a sysimage is

A native shared library bundling already-compiled OpenJFEM code and its dependencies. Julia loads it instead of compiling at startup, eliminating most of the first-run latency.

Build (Windows):

```
julia --threads=auto --startup-file=no --project=. ‘
      .\tools\deploy_fast.jl ‘
      --deck C:\models\VTP105.bdf ‘
      --sysimage=build\OpenJFEM_sysimage.dll ‘
      --install-packagecompiler
```

Build (Linux helper):

```
JFEM_SYSIMAGE /projects/.../MODELS/VTP105.bdf
```

Sysimage: when to (re)build it

Rebuild required

- JFEM source code changed
- `Project.toml` / `Manifest.toml` changed
- Julia package update
- Different OS / CPU architecture
- Representative workload changed substantially

Reuse is safe

- Running new decks of the same SOL
- Output toggles changed
- `JFEM_*` env flags changed
- Different mesh sizes
- Different material data

Default sysimage path

Linux helpers: `<jfem_dir>/build/OpenJFEM_sysimage.so`
Windows: typically `build\OpenJFEM_sysimage.dll`

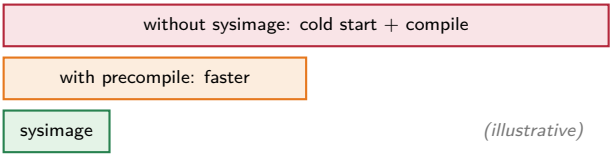
Running with a sysimage

Windows (single case):

```
julia --sysimage build\OpenJFEM_sysimage.dll ‘
     --threads=auto --startup-file=no ‘
     --project=. .\tools\testing\run_bdf.jl ‘
     C:\models\panel.bdf D:\runs\panel ‘
     "JFEM_EXPORT_BINARY=false,JFEM_SUPPRESS_THREAD_HINT=1"
```

Linux (helper):

```
JFEM_SYS /projects/.../MODELS/panel.bdf /runs/panel
```



Single-case launcher `tools/testing/run_bdf.jl`

Signature:

```
julia ... run_bdf.jl <input.bdf> <output_dir> [FLAG=val,FLAG2=val2]
```

Behavior:

- Creates `output_dir` if missing.
- Writes a `run_manifest.json` capturing the launch parameters.
- Calls `OpenJFEM.main(BDF_PATH; output_dir=OUT_DIR, export_jfem_binary=...)`.
- Honors `JFEM_EXPORT_BINARY` from the flag string.

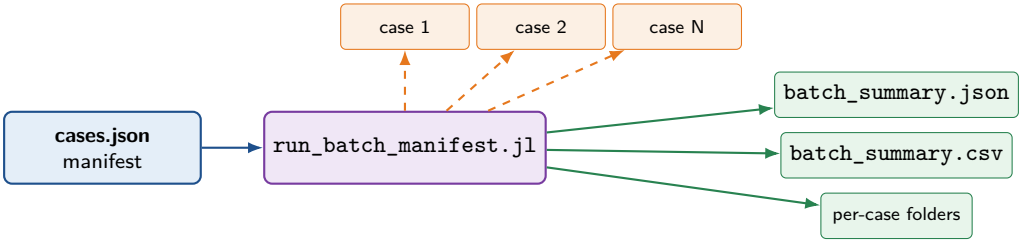
Example (Windows):

```
julia --threads=auto --startup-file=no --project=. ‘
     .\tools\testing\run_bdf.jl ‘
     C:\models\panel_001.bdf D:\runs\panel_001 ‘
     "JFEM_EXPORT_BINARY=false,JFEM_MATRIX_ASYMMETRY_CHECK=false,JFEM_SOL105_STORE_PUBLIC_MODE_SHAPES=false,
          JFEM_SUPPRESS_THREAD_HINT=1"
```

Example (Linux helper):

```
JFEM /projects/.../panel_001.bdf /runs/panel_001
JFEM_SYS /projects/.../panel_001.bdf /runs/panel_001
```

Batch runs — JSON manifest is the preferred interface



Why a manifest beats shell loops

Every input deck, output folder, flag, and output toggle is explicit. The same JSON file feeds command-line, Python loops, and job schedulers, and avoids quoting bugs.

Minimal SOL 105 manifest

```
{
  "batch_id": "batch_001",
  "output_root": "D:/jfem_runs/batch_001",
  "defaults": {
    "flags": {
      "JFEM_EXPORT_BINARY": "false",
      "JFEM_MATRIX_ASYMMETRY_CHECK": "false",
      "JFEM_SOL105_STORE_PUBLIC_MODE_SHAPES": "false",
      "JFEM_SUPPRESS_THREAD_HINT": "1"
    },
    "output_options": {
      "binary": false,
      "json": true,
      "report": true
    },
    "gc_between": true,
    "stop_on_error": false
  },
  "cases": [
    { "case_id": "panel_001", "input": "C:/models/panel_001.bdf",
      "output_dir": "D:/jfem_runs/batch_001/panel_001" },
    { "case_id": "panel_002", "input": "C:/models/panel_002.bdf",
      "output_dir": "D:/jfem_runs/batch_001/panel_002" }
  ]
}
```

/ works as path separator on Windows too — avoids escaping backslashes.

Manifest reference — fields and effects

Field	Meaning
`batch_id`	Readable name; appears in summaries.
`output_root` *(required)*	Batch-level output folder.
`defaults.flags`	Map of `JFEM_*` env values applied to every case.
`defaults.output_options.binary`	Toggle `.jfem` export per case.
`...output_options.json`	Toggle results JSON.
`...output_options.vtk`	Toggle ParaView `.vtu`.
`...output_options.hdf5`	Toggle HDF5.
`...output_options.report`	Markdown report (default `true`).
`...output_options.eigenvalues_only`	SOL 105 — discard eigenvectors.
`...output_options.eigenvectors`	SOL 105 — keep mode shapes.
`defaults.gc_between`	Force GC between cases (memory hygiene).
`defaults.stop_on_error`	Abort batch on first failure.
`cases[].case_id`	Stable name in summaries; can repeat slug with suffix.
`cases[].input` *(required)*	`.bdf`/`.dat`/`.nas` path.
`cases[].output_dir`	Custom output folder (otherwise `output_root/slug`).
`cases[].flags` / `cases[].output_options`	Per-case overrides.

Running a batch manifest

Windows:

```
julia --threads=auto --startup-file=no --project=. ‘
     .\tools\run_batch_manifest.jl C:\models\cases.json --quiet
```

Linux:

```
julia --threads=auto --startup-file=no --project=. \
     ./tools/run_batch_manifest.jl /home/user/models/cases.json --quiet
```

Useful flags on the batch runner:

- `--quiet` keeps per-case solver output out of stdout; each case's chatter goes to `<case>/jfem_case_stdout.log`.
- `--stop-on-error` aborts the batch on the first failure (overrides manifest setting).

Outputs written:

```
<output_root>/batch_summary.csv
<output_root>/batch_summary.json
<output_root>/<case_id>/run_manifest.json
<output_root>/<case_id>/<input_stem>.REPORT.md
<output_root>/<case_id>/<input_stem>.JU.JSON # if json=true
<output_root>/<case_id>/jfem_case_stdout.log
```

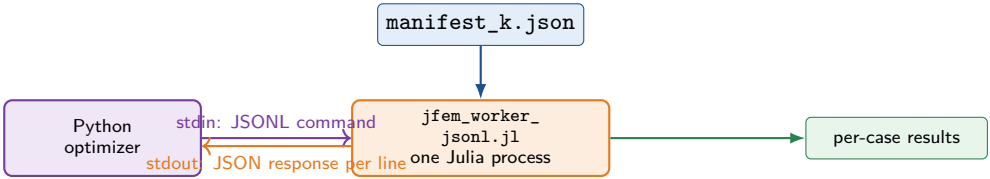
Legacy text/CSV batch (`run_bdf_batch.jl`)

Argument / option	Meaning
`<input>` (positional)	One `.bdf`, a directory of decks, or a text/CSV list (one path per line).
`<output_root>` (positional)	Top-level output folder.
`FLAG=val,...` (positional)	Flag string, same shape as single-case runner.
`--recursive`	Walk subdirectories.
`--ext=.bdf,.dat`	Override accepted deck extensions.
`--stop-on-error`	Stop on the first failed case.
`--dry-run`	List cases without solving.
`--no-gc-between`	Skip the explicit GC step between cases.

When to use which batch tool

New automation: use the JSON manifest (`run_batch_manifest.jl`).
Quick ad-hoc sweep over a folder of decks: `run_bdf_batch.jl` is fine.
Python sizing loops: prefer the JSONL worker (next slide).

Persistent JSONL worker for Python optimization loops



Why keep Julia open

Restarting Julia per design iteration pays minutes of compile/load each time. The worker stays loaded, accepts JSONL commands, and runs manifests in a single process — the recommended architecture for hot Python sizing loops.

JSONL commands accepted: `status` / `ping`, `gc`, `run_batch` (with `manifest`), `quit`.

Starting the JSONL worker

Windows:

```
julia --threads=auto --startup-file=no --project=. ‘
      .\tools\jfem_worker_jsonl.jl
```

Linux:

```
julia --threads=auto --startup-file=no --project=. \
      ./tools/jfem_worker_jsonl.jl
```

Example exchange on stdin/stdout:

```
{"command":"status"}
{"command":"run_batch","manifest":"C:/models/iter_007.json"}
{"command":"gc"}
{"command":"quit"}
```

stdout discipline: stdout is reserved for JSON responses (one per line). Solver chatter is written to `jfem_case_stdout.log` in each case's output folder; human-readable diagnostics go to stderr.

Recommended recipes by user type

Engineer / quick run

1. `JFEM_PRECOMPILE` (once)
2. `JFEM deck.bdf`
3. Open `deck.REPORT.md`

No sysimage. Defaults are fine.

Production batch

1. `JFEM_SYSIMAGE rep.bdf`
2. Write `cases.json`
3. `run_batch_manifest.jl`

Manifest = single source of truth.

Python sizing loop

1. `JFEM_SYSIMAGE rep.bdf`
2. Start `jfem_worker_jsonl.jl`
3. Stream `run_batch` commands

One Julia process for thousands of iters.

Debugging a deck

1. `JFEM deck.bdf`
2. Inspect `deck.bdf.json`
3. Inspect `deck.REPORT.md`

Use `export_model_json=true`.

Visual review

1. Run with `export_vtk=true`
2. Open `.vtu` in ParaView
3. Or open `.jfem` in POST viewer

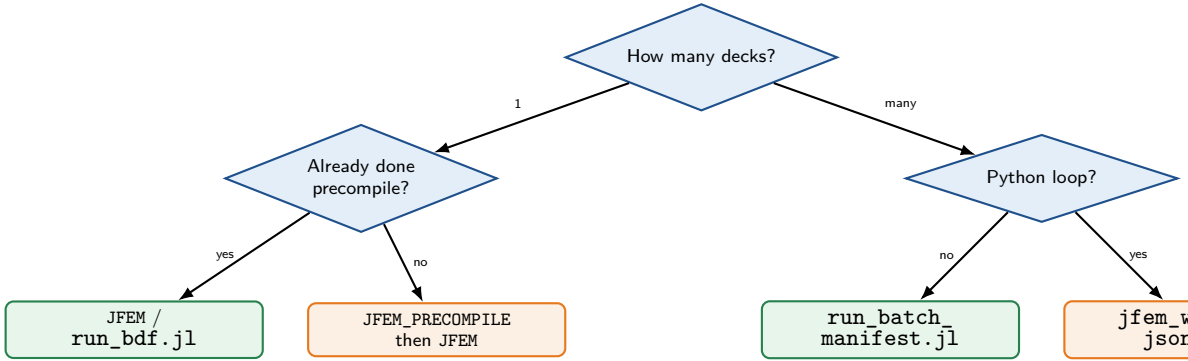
POST/postv11.html is browser-based.

Sensitivities

1. Place `adjoint_config.json` next to the deck
2. Run normally
3. Read `.ADJOINT.JSON`

Triggered after the forward solve.

Decision tree: which launcher should I use?



Add a sysimage when

You will launch the solver many times *and* restart Julia between launches (e.g., HPC job scripts, shell-driven sweeps). For a single long-lived Julia process, the JSONL worker already amortizes startup — a sysimage is optional.

Common pitfalls and fixes

Symptom	Likely cause / fix
`Error: JFEM sysimage not found`	The sysimage path does not exist. Run `JFEM_SYSIMAGE` (Linux) or `deploy_fast.jl --sysimage=...` (Windows).
First run takes minutes	Julia is compiling. Run `deploy_fast.jl` once after install; later runs are fast.
`'\' is not recognized` on Windows	PowerShell line-continuation is backtick ‘, not backslash. Or put the command on one line.
`Error: ... does not look like a Julia project`	`--project=.` was run outside the JFEM repo. `cd` into the repo first, or use `--project=C:\JFEM`.
Solver runs but no `.jfem` appears	`JFEM_EXPORT_BINARY=false` or `binary:false` in the manifest. Flip the toggle.
Batch run silent in console	`--quiet` routes per-case chatter to `jfem_case_stdout.log`.
Sensitivities not produced	`adjoint_config.json` must sit next to the input deck, not in the output folder.
Wrong Julia picked up	Set `JULIA=/path/to/julia` before calling helpers.
Slow buckling diagnostic	Set `JFEM_MATRIX_ASYMMETRY_CHECK=false`.
Memory growth in long batches	Keep `defaults.gc_between=true`; the worker also accepts a `{"command":"gc"}`.

Quick reference card

Task	Command (short form)
First-time precompile	`julia ... .\tools\deploy_fast.jl` *or* `JFEM_PRECOMPILE`
Precompile + sysimage	`... deploy_fast.jl --deck rep.bdf --sysimage=... --install-packagecompiler` *or* `JFEM_SYSIMAGE rep.bdf`
Single run	`... run_bdf.jl deck.bdf out_dir "flags"` *or* `JFEM deck.bdf`
Single run + sysimage	`julia --sysimage img.dll ... run_bdf.jl ...` *or* `JFEM_SYS deck.bdf`
Batch manifest	`... run_batch_manifest.jl cases.json --quiet`
Text/dir batch	`... testing\run_bdf_batch.jl <input> <out_root>`
Python loop	`... jfem_worker_jsonl.jl` (one process, JSONL on stdin)

Three numbers to remember

1: first `deploy_fast.jl` after install. **4**: Linux helpers (`JFEM`, `JFEM_PRECOMPILE`, `JFEM_SYSIMAGE`, `JFEM_SYS`). **6**: output toggles (`jfem_binary`, `report`, `json`, `vtk`, `hdf5`, `model_json`).

One-page cheat sheet

Windows — single SOL 105 case end-to-end:

```
cd C:\JFEM
julia --threads=auto --startup-file=no --project=. .\tools\deploy_fast.jl --deck C:\models\VTP105.bdf --sysimage=
    build\OpenJFEM_sysimage.dll --install-packagecompiler
julia --sysimage build\OpenJFEM_sysimage.dll --threads=auto --startup-file=no --project=. .\tools\testing\run_bdf.jl
    C:\models\panel.bdf D:\runs\panel "JFEM_EXPORT_BINARY=false,JFEM_MATRIX_ASYMMETRY_CHECK=false,
    JFEM_SOL105_STORE_PUBLIC_MODE_SHAPES=false,JFEM_SUPPRESS_THREAD_HINT=1"
```

Linux — same scenario:

```
source /path/to/jfem_linux_bash_functions.sh
JFEM_SYSIMAGE /projects/.../MODELS/VTP105.bdf
JFEM_SYS /projects/.../MODELS/panel.bdf /runs/panel
```

Batch (cross-platform):

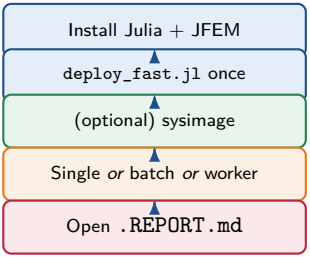
```
julia --threads=auto --startup-file=no --project=. ./tools/run_batch_manifest.jl ./cases.json --quiet
```

Python loop:

```
julia --threads=auto --startup-file=no --project=. ./tools/jfem_worker_jsonl.jl # then feed JSONL on stdin
```

Summary

- One launcher idea: `julia + project + script + deck + flags`.
- Windows and Linux share the *same* Julia commands; the Linux helpers just wrap them in four function names.
- Run `deploy_fast.jl` **once** after install — saves minutes per cold start.
- Build a sysimage *only* when you will spawn many short-lived Julia processes.
- Use the **JSON manifest** for batches; use the **JSONL worker** for hot Python loops.
- Toggle outputs deliberately: `.jfem` off for speed, on for archival.



One workflow, two operating systems, four launch modes.

Happy solving!

Run `deploy_fast.jl` once. Launch with confidence.